Tribhuvan University

Faculty of Humanities and Social Sciences

A PROJECT REPORT

ON

MAZEMASTER: PATHFINDER'S ADVENTURE GAME USING BFS

Submitted To

Department of Computer Application

College Name

In partial fulfillment of the requirements for the Bachelors in Computer Application

Submitted By

Dhiraj Jirel

Symbol No: 1059xxxxx

August 2025

Under the supervision of

Ms. Supervisor Name

Tribhuvan University

Faculty of Humanities and Social Science

College Name

Supervisor's Recommendation

I hereby recommend that this project prepared under my supervision by **DHIRAJ JIREL** entitled **"MAZEMASTER: PATHFINDER'S ADVENTURE GAME USING BFS"** in partial fulfillment of the requirements for the degree of Bachelor of Computer Application is recommended for the final evaluation.

.....

SIGNATURE

Ms. Supervisor Name

Supervisor

College Name

College Full Address

Tribhuvan University

Faculty of Humanities and Social Science

College Name

Letter of Approval

This is to certify that this project prepared by **DHIRAJ JIREL** entitled **"MAZEMASTER: PATHFINDER'S ADVENTURE GAME USING BFS"** in partial fulfillment of the requirements for the degree of Bachelor's in Computer Application has been evaluated. In our opinion it is satisfactory in the scope and quality as a project for the required degree.

…… **Ms. Supervisor Name** **Supervisor** College Name College Full Address	…….. **Mr. Coordinator Name** **Coordinator** College Name College Full Address
…….. **Internal Examiner**	…….. **External Examiner**

Abstract

MazeMaster: Pathfinder's Adventure Game is an engaging web-based game designed to provide an interactive maze-solving experience, combining manual navigation with algorithmic pathfinding. The primary goal of this project is to create a fun and competitive platform that introduces players to the Breadth-First Search (BFS) algorithm. By incorporating this algorithm into gameplay, the game not only entertains but also educates users on how BFS can efficiently find the shortest path in a maze. In this project, players tackle 15 carefully crafted mazes divided into Easy, Medium, and Hard difficulty levels. They can navigate using keyboard controls or utilize the BFS algorithm, limited to three uses per level, to discover the shortest path. Registered users benefit from features such as progress saving, score tracking, and access to a global leaderboard for competitive play. Public users, however, are restricted to Easy levels and cannot save their progress. The game is built using modern technologies that is Angular powers the dynamic front-end interface, Spring Boot handles the back-end logic, and PostgreSQL manages the database, providing a smooth and efficient user experience.

Keywords: *BFS Algorithm, Angular, Spring Boot, PostgreSQL*

Acknowledgements

I sincerely acknowledge and thank those who have contributed their valuable time in helping me to achieve success in my project work. I want to thank **College Name and Tribhuvan University** for giving me the wonderful opportunity for assigning the project where I got opportunities to enhance my skills.

I am indebted and thankful to my Project Supervisor, **Ms. Supervisor Name** to whom I owe his piece of knowledge for his valuable and timely guidance, co-operation, encouragement and time spent for doing this project work. I am immensely obliged to my friends for their elevating inspiration, encouraging guidance and kind support in the completion of my project.

Last, but not the least, my parents are also an important inspiration for me. So, with due regards, I express my gratitude to them.

Sincerely,
Dhiraj Jirel (1059xxxxx)

Table of Contents

List of Abbreviations

Abbreviation	Definition
2D	Two-Dimensional
A* Algorithm	A-star Algorithm
ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
BFS	Breadth-First Search
DB	Database
DFS	Depth-First Search
HTTPS	Hypertext Transfer Protocol Secure
IDE	Integrated Development Environment
JWT	JSON Web Token
MS Word	Microsoft Word
NPC	Non-Player Character
OTP	One-Time Password
URL	Uniform Resource Locator

List of Figures

List of Tables

Chapter 1: Introduction

1.1 Introduction

In the world of interactive digital entertainment, maze-solving games have long captivated players with their blend of strategy and exploration. MazeMaster: Pathfinder's Adventure Game is a web-based platform designed to elevate this classic experience by integrating the BFS algorithm. Breadth First Search (BFS) is a fundamental graph traversal algorithm**.** It begins with a node, then first traverses all its adjacent nodes. Once all adjacent are visited, then their adjacent are traversed [1].

The game features 15 pre-generated mazes across three difficulty levels: Easy, Medium, and Hard. Each with five sub-levels, offering diverse challenges for players of varying skill levels. This game allows users to navigate mazes manually using keyboard inputs or to employ the BFS algorithm, limited to three uses per level, to discover the shortest path.

By blending entertainment with education, MazeMaster makes complex computer science concepts accessible and enjoyable, drawing inspiration from research on pathfinding in game development [2] and puzzle-solving applications [3].

1.2 Problem Statement

The primary challenge addressed by the project is to create an interactive learning platform that makes complex computer science concepts, such as pathfinding algorithms, accessible and enjoyable for a broad audience. Traditional methods of teaching algorithms often rely on theoretical explanations and static examples, which can be less engaging and harder to grasp for many learners. This project aims to bridge this gap by providing a hands-on, game-based learning experience where players can experiment with BFS in a practical setting.

1.3 Objectives

The main objectives of the project are:

- To develop an engaging web-based game where players can manually solve mazes or use the BFS algorithm to find solutions.
- To support user accounts for tracking progress and scores, and to feature a global leaderboard.

- To ensure that the game is accessible to both registered users and public, with appropriate limitations for public users.

1.4 Scope and Limitation

1.4.1 Scope

- The game includes 15 pre-generated mazes across three difficulty levels.
- It supports both manual and automated (using BFS) solving methods.
- User accounts allow saving progress, scores, and leaderboard participation.
- The game is web-based, accessible via standard web browsers.

1.4.2 Limitations

- The game is limited to teaching the BFS algorithm and does not cover other pathfinding algorithms.
- Maze generation is pre-defined and not dynamic.
- Public players are restricted to Easy levels and cannot save progress.

1.5 Development Methodology

The development methodology of this project followed an iterative methodology. It is a development approach where a project is built in repeated cycles, with each cycle (or iteration) improving upon the previous one [4].

The project was broken down into smaller, manageable tasks, with regular reviews and adjustments based on feedback and testing. The development process included:

- Requirement gathering and analysis.
- System design, including architectural, database, and interface design.
- Implementation using appropriate programming languages and tools.
- Testing at various stages.
- Deployment and maintenance.

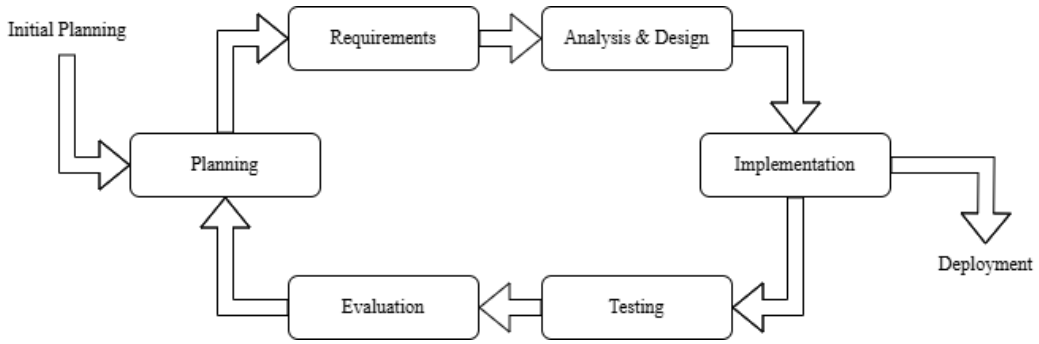


Figure 1.1 Iterative approach Diagram

1.6 Report Organization

Chapter 1: Introduction

This chapter provides an overview of the project, problem statement, objectives, scope, development methodology, and report organization.

Chapter 2: Background Study and Literature Review

This chapter discusses the fundamental theories and concepts related to pathfinding algorithms and reviews similar projects.

Chapter 3: System Analysis and Design

This chapter provide details on the system requirements, feasibility, modeling, and design aspects.

Chapter 4: Implementation and Testing

This chapter describes the tools used, implementation details, and testing procedures.

Chapter 5: Conclusion and Future Recommendations

The final chapter summarizes the project outcomes, lessons learned, and suggestions for future enhancements.

Chapter 2: Background Study and Literature Review

2.1 Background Study

Pathfinding is a fundamental problem in computer science and artificial intelligence, involving the search for a path from a starting point to a goal in a given space. In the context of this project, the space is a maze, which can be represented as a graph where each cell is a node, and possible moves between cells are edges [1].

Breadth-First Search (BFS) is a pathfinding algorithm that explores all possible paths level by level, starting from the initial node. It uses a queue to keep track of nodes to visit next. The algorithm proceeds as follows:

1. Enqueue the starting node.
2. While the queue is not empty:
   (a) Dequeue a node.
   (b) If this node is the goal, stop and return the path.
   (c) Otherwise, enqueue all its unvisited neighbors.
3. If the queue becomes empty without finding the goal, there is no path.

BFS guarantees the shortest path in terms of the number of edges traversed, making it suitable for unweighted graphs like mazes where each move has the same cost [5]. This property is critical for maze-solving games, where players seek efficient routes.

In game development, BFS is used for Non-Player Character (NPC) navigation and level design, though its computational intensity can be a drawback in large mazes, prompting the use of more efficient algorithms like A* [2]. Despite this, BFS's simplicity and optimality make it a valuable tool for educational games like MazeMaster, enabling players to grasp pathfinding concepts interactively.

2.2 Literature Review

The application of pathfinding algorithms, particularly BFS, in educational games and maze-solving has been widely studied. Luthfisauqi's article, "Artificial Intelligence: Search Problem: Solve Maze Using Breadth-First Search (BFS) Algorithm," explains how BFS efficiently finds the shortest path in unweighted maze graphs, emphasizing its practical implementation [6]. GeeksforGeeks provides code examples, such as "Shortest Path in a Binary Maze" and "Count Number of Ways to Reach Destination in a Maze using BFS," showcasing BFS's versatility in finding paths and counting solutions [7].

A research paper on puzzle game solving demonstrates BFS's effectiveness in solving a 3x3 puzzle in under 4 seconds, highlighting its ability to avoid deadlocks and find the shortest solution, though it notes high memory usage and potential time consumption for larger problems [3]. A review of pathfinding in game development compares BFS to A*, noting that while BFS ensures optimality, it is less efficient due to its exhaustive exploration, with efforts like graph clustering aimed at improving performance [2].

These studies affirm BFS's educational value and practical application in games, despite its computational trade-offs. MazeMaster leverages these insights to create an engaging platform for learning BFS.

Chapter 3: System Analysis and Design

3.1 System Analysis

This chapter discusses about the proposed design of the system, requirement analysis, requirement specification. It also talks about the functional requirements of the system, the system design. The application architecture of the system will also be shown in this chapter, the use case diagram, object and class diagram, activity diagrams, state and sequence diagram and system design will all be shown in this chapter.

3.1.1 Requirement Analysis

The following section presents the complete set of functional requirements:

i. Functional Requirements

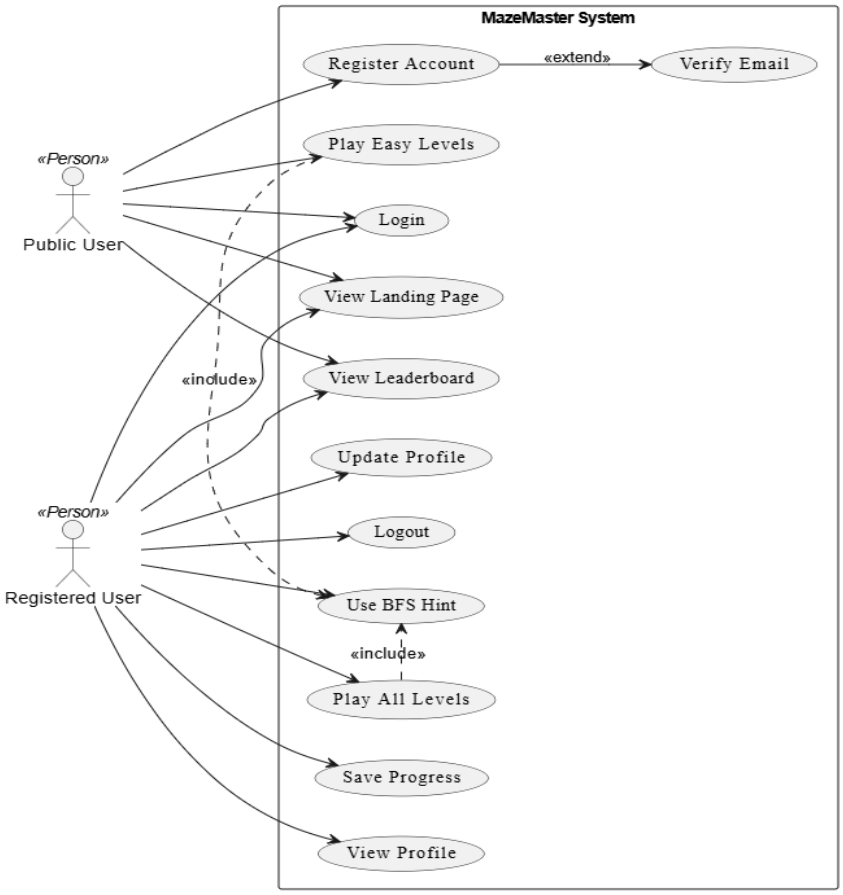


Figure 3.1 Use Case Diagram of MazeMaster

Use Case Description:

Table 3.1 Use Case: View Landing Page

Item	Description
Use Case ID	UC1
Actors	Public User, Registered User
Description	Allows any user to view the landing page with general information about the game.
Preconditions	None
Post conditions	Landing page is displayed
Main Flow	1. User accesses the home URL 2. Landing page loads successfully
Alternate Flows	- If server error, show "Unable to load page" message

Table 3.2 Use Case: Play Easy Levels

Item	Description
Use Case ID	UC2
Actors	Public User
Description	Allows public users to play easy-level mazes.
Preconditions	None
Post conditions	Game starts with an easy maze
Includes	Use BFS Hint (UC9)
Main Flow	1. Public user clicks “Play Easy” 2. Maze loads 3. User optionally uses BFS hint 4. User plays maze
Alternate Flows	- If maze fails to load, an error is shown

Table 3.3 Use Case: View Leaderboard

Item	Description
Use Case ID	UC3
Actors	Public User, Registered User
Description	Displays the top players' scores and rankings.
Preconditions	Leaderboard data exists
Post conditions	Leaderboard is displayed
Main Flow	1. User clicks "Leaderboard" 2. Rankings and usernames are shown
Alternate Flows	- If no data, show "No scores yet" message

Table 3.4 Use Case: Register Account

Item	Description
Use Case ID	UC4
Actors	Public User
Description	Allows new users to register by providing email, username, and password.
Preconditions	User must provide valid inputs
Post conditions	Account is created in pending verification status
Extends	Verify Email (UC6)
Main Flow	1. User enters registration info 2. System saves user and sends OTP
Alternate Flows	- If email exists, show "Email already in use"

Table 3.5 Use Case: Verify Email

Item	Description
Use Case ID	UC6
Actors	Public User
Description	Verifies email via OTP sent to the user's email.
Preconditions	User has registered
Post conditions	User account is activated

Main Flow	1. User entered the OTP from the email 2. Account is verified
Alternate Flows	- Invalid OTP, show error

Table 3.6 Use Case: Login

Item	Description
Use Case ID	UC5
Actors	Public User, Registered User
Description	Allows a user to login using email and password.
Preconditions	User has a verified account
Post conditions	JWT Token is created
Main Flow	1. User enters credentials 2. System validates and logs in user
Alternate Flows	- Wrong password, show error- Account unverified, show "Verify your email"

Table 3.7 Use Case: Play All Levels

Item	Description
Use Case ID	UC7
Actors	Registered User
Description	Allows full access to easy, medium, and hard maze levels.
Preconditions	User is logged in
Post conditions	Game starts with selected difficulty maze
Includes	Use BFS Hint (UC9)
Main Flow	1. User selects difficulty 2. Maze loads 3. User optionally uses BFS hint 4. User plays
Alternate Flows	- Server/maze error, show appropriate message

Table 3.8 Use Case: Save Progress

Item	Description
Use Case ID	UC8
Actors	Registered User
Description	Allows saving game progress for resumption later.
Preconditions	User is logged in and currently playing
Post conditions	Game state is saved in DB
Main Flow	1. User clicks “Next Level” 2. Progress is stored
Alternate Flows	- Server error, notify user of failure

Table 3.9 Use Case: Use BFS Hint

Item	Description
Use Case ID	UC9
Actors	Registered User, Public User
Description	Provides a BFS-based path hint on current maze.
Preconditions	User is playing a maze
Post conditions	Hint is shown on maze
Main Flow	1. User clicks “Use BFS” 2. BFS logic shows optimal path
Alternate Flows	- Maze not loaded, show "No maze active"

Table 3.10 Use Case: View Profile

Item	Description
Use Case ID	UC10
Actors	Registered User
Description	Displays user's profile including progress, levels completed, etc.
Preconditions	User is logged in
Post conditions	Profile data is displayed
Main Flow	1. User clicks “Profile”

	2. Data is fetched and shown
Alternate Flows	- Profile fetch fails, show error

Table 3.11 Use Case: Logout

Item	Description
Use Case ID	UC11
Actors	Registered User
Description	Logs the user out of the system.
Preconditions	User is logged in
Post conditions	Token Expired
Main Flow	1. User clicks “Logout” 2. JWT Token is terminated 3. Redirected to landing page
Alternate Flows	- If already logged out, do nothing

ii. Non-functional Requirements

Some of the contents of non-functional requirements are shown table below:

1. **Accessibility**
   - The game shall be fully web-based and accessible via standard web browsers such as Chrome, Firefox, Edge, and Safari.
   - Users should not require additional plugins or installations to access and play the game.
2. **Security**
   - Secure login should be enforced for all registered users, using industry-standard authentication protocols.
   - Sensitive user data, including email, password, and progress history, must be securely encrypted both in transit (HTTPS) and at rest.
   - The system must implement role-based access control (e.g., public users vs. registered users vs. administrators).

3. **Maintainability**
    - The application codebase should be modular and follow clean coding practices to ease debugging and future updates.
    - The system should support automated backups of user progress and leaderboard data.
    - Logging and monitoring tools should be integrated to assist with issue diagnosis and system performance tracking.

3.1.2 Feasibility Analysis

Some important feasibility studies are mentioned below:

i. Technical Feasibility

Developing a web-based game with the described features is technically feasible using modern web technologies such as frontend framework like Angular, and a backend framework like Spring Boot with a database like PostgreSQL. Implementing BFS for maze solving is straightforward, as it is a well-understood algorithm.

ii. Operational Feasibility

The game can be hosted on a web server, and users can access it via the internet. Maintenance would involve updating the server, fixing bugs, and possibly adding new features or levels. Training users is minimal since the game is designed to be intuitive, but documentation and tutorials can be provided.

iii. Economic Feasibility

Initial development costs would include developer time, server hosting, and any third-party services (e.g., for user authentication). Ongoing costs would be for server maintenance and potential updates. Revenue could be generated through advertisements, premium features, or depending on the business model.

3.1.3 Object Modelling: Object & Class Diagram

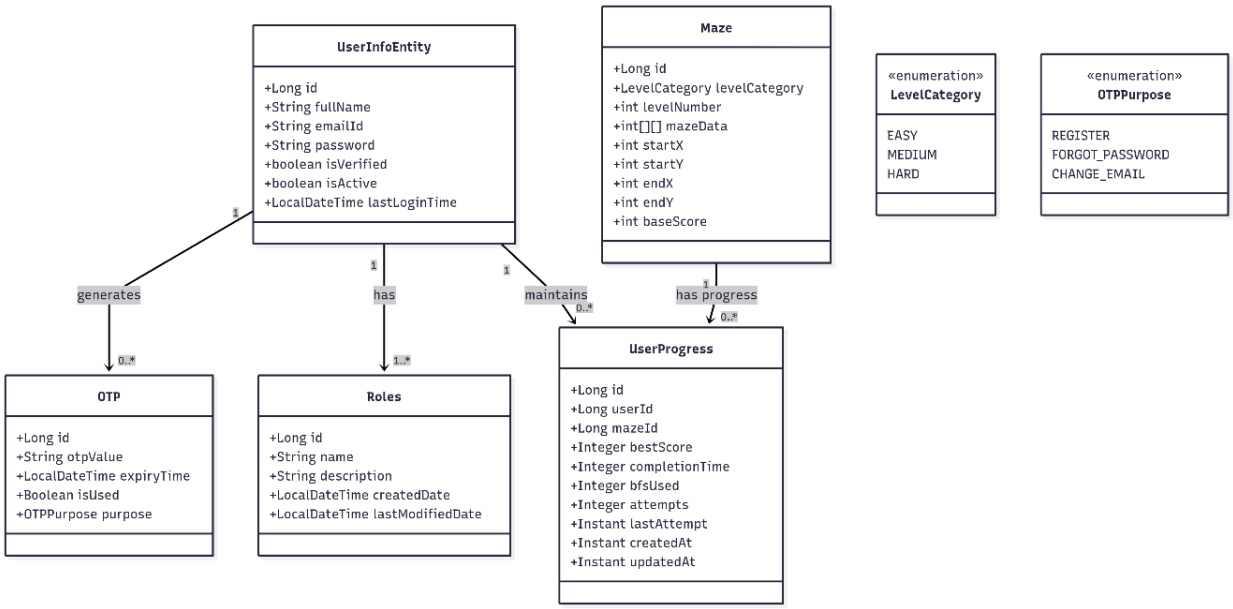


Figure 3.2 Object & Class Diagram

3.1.4 Dynamic Modelling: State & Sequence diagram

1. State Diagram for User Authentication:

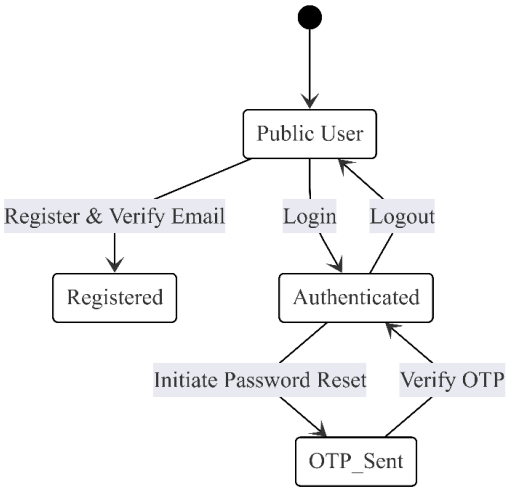


Figure 3.3 State Diagram for User Authentication

2. Sequence Diagram - Level Completion

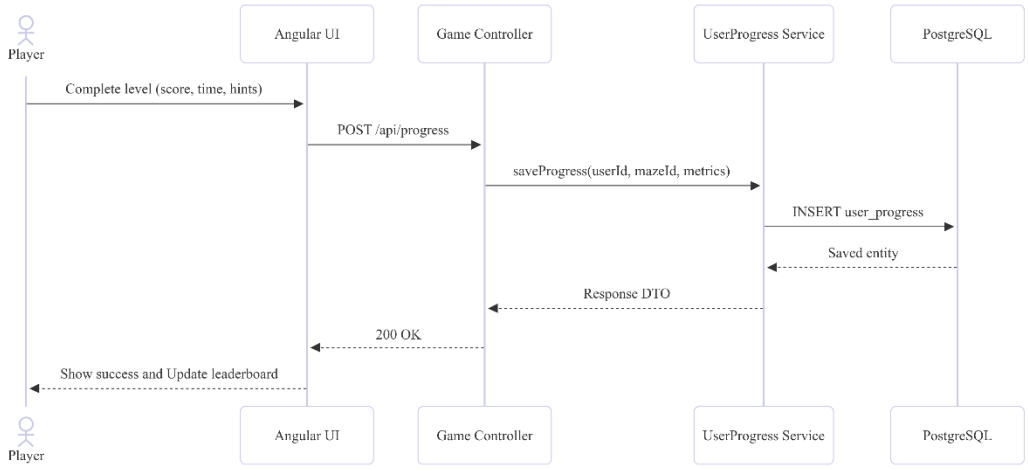


Figure 3.4 Sequence Diagram - Level Completion

3.1.5 Process modelling: Activity Diagram

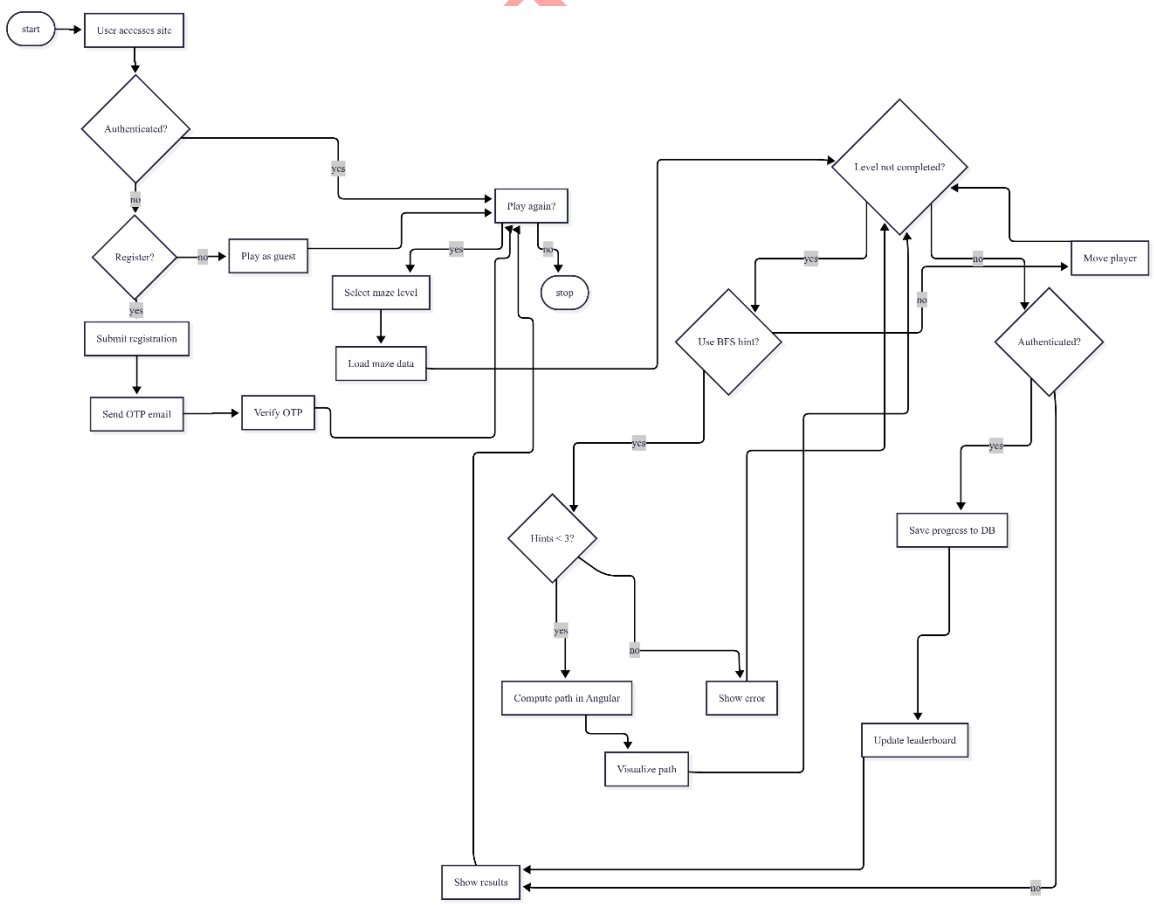


Figure 3.5 Activity Diagram

3.2 System Design

3.2.1 Refinement of Classes and Object

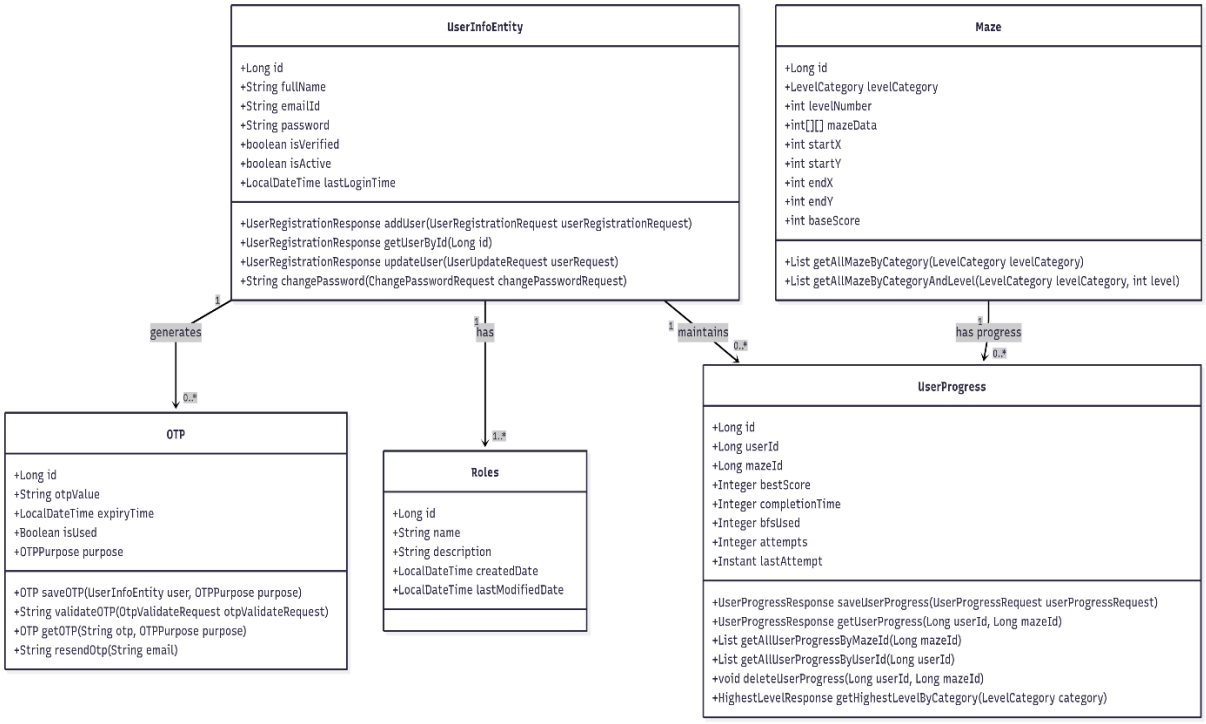


Figure 3.6 Refinement of Classes and Object Diagram

3.2.2 Component diagram

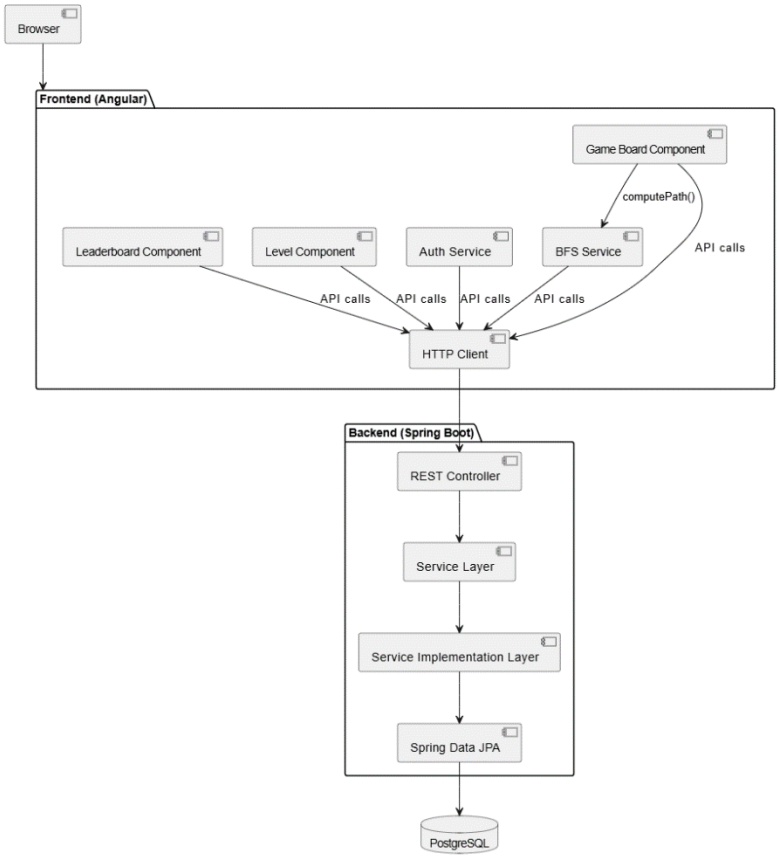


Figure 3.7 Component Diagram of MazeMaster

3.2.3 Deployment diagram

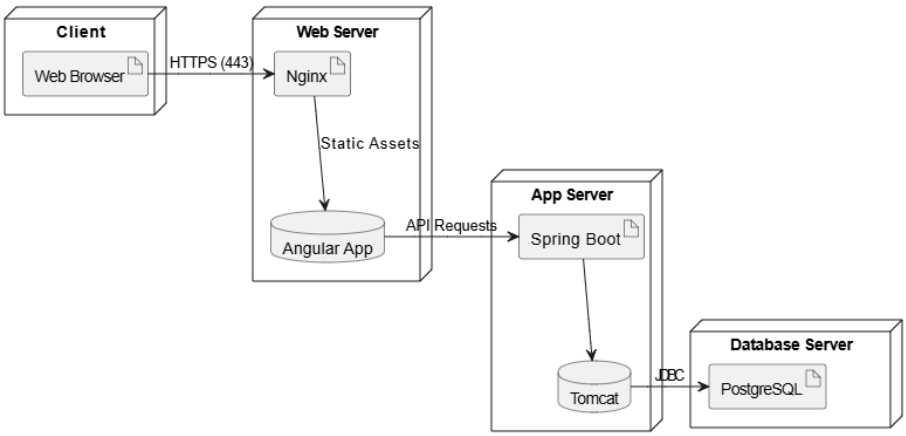


Figure 3.8 Deployment Diagram of MazeMaster

3.3 Algorithm Details

The key algorithm used in MazeMaster is Breadth-First Search (BFS) for finding the shortest path in the maze. Below is a detailed explanation of its implementation, including maze generation and scoring.

BFS Algorithm for Maze Solving:

1. Represent the maze as a 2D grid, where each cell can be either a wall or a path.
2. Define the start and end positions.
3. Initialize a queue and enqueue the start position.
4. Mark the start position as visited.
5. While the queue is not empty:
    a. Dequeue the current position.
    b. If the current position is the end, reconstruct and return the path.
    c. For each adjacent cell (up, down, left, right):
        i. If it is within bounds, is a path, and not visited:
            1. Enqueue it.
            2. Mark it as visited.
            3. Set its parent to the current position (for path reconstruction).
6. If the queue is empty and the end is not found, there is no path.

In the game, when the player chooses to use BFS, the algorithm is run from the current position to the end, and the path is highlighted or the player is moved along the path.

Maze Generation

Mazes are pre-generated with sizes based on difficulty (e.g., 6x6 for Easy, 10x10 for Hard). The InsertMazeData class uses BFS to ensure a valid path exists between start and end positions, then adds random walls with probabilities (e.g., 20% for Easy, 40% for Hard) to increase complexity while preserving solvability.

Example Implementation

Below is the Java implementation for BFS-based maze generation:

```
private List<int[]> createPathWithBFS(List<List<Integer>> maze, int startX, int startY, int endX, int endY) {
    int size = maze.size();
    boolean[][] visited = new boolean[size][size];
```

```
int[][] parent = new int[size * size][2];
Queue<int[]> queue = new LinkedList<>();
queue.add(new int[]{startX, startY});
visited[startX][startY] = true;
parent[startX * size + startY] = new int[]{-1, -1};

int[][] directions = {{-1, 0}, {1, 0}, {0, -1}, {0, 1}};

while (!queue.isEmpty()) {
    int[] current = queue.poll();
    int x = current[0];
    int y = current[1];

    if (x == endX && y == endY) {
        break;
    }

    for (int[] dir : directions) {
        int newX = x + dir[0];
        int newY = y + dir[1];
        if (isValidMove(maze, newX, newY, visited)) {
            queue.add(new int[]{newX, newY});
            visited[newX][newY] = true;
            parent[newX * size + newY] = new int[]{x, y};
        }
    }
}

List<int[]> path = new LinkedList<>();
int[] current = new int[]{endX, endY};
while (current[0] != -1) {
    path.add(0, current);
    current = parent[current[0] * size + current[1]];
}
```

```
    for (int[] cell : path) {
        maze.get(cell[0]).set(cell[1], 0);
    }
    return path;
}
```

Scoring System

The scoring system balances maze difficulty, time, and BFS usage:

[final Score = base Score – time Penalty – BFS Penalty]

Parameter	Description	Formula
Base Score	Varies by difficulty (e.g., 100 for Easy)	Depends on level category and number
Time Penalty	1 point per second	(time Elapsed / 1000)
BFS Penalty	10% of base score per use	(base Score * 0.1 * BFS Used)
Final Score	Non-negative score	(max(0, final Score))

This system encourages efficient play, rewarding manual navigation while allowing strategic BFS use.

Chapter 4: Implementation and Testing

4.1 Implementation

The aim of this chapter is to document the process of development of the main features. It gives a detailed breakdown of the problems encountered and how they were resolved. It also goes through the test plan and test report of the project to ensure all the functionalities are functioning properly. This chapter is where the project is going to be implemented. However, the software and hardware components used in the implementation of this project will be analyzed below.

4.1.1 Tools Used

Frontend Tools

Angular: A TypeScript-based, open-source front-end framework used to develop the dynamic and responsive user interface. It is component-based architecture and two-way data binding facilitated the creation of interactive maze navigation and BFS visualization features, ensuring a seamless user experience.

Visual Studio Code (VSCode): A lightweight, versatile code editor used for front-end development with Angular. It has extensive plugin ecosystem, including support for TypeScript and Angular extensions, streamlined coding, testing, and debugging of the user interface.

Backend Tools

Spring Boot: A Java-based framework employed for the back-end development. Spring Boot's has support and built-in security features enabled efficient handling of game logic, user authentication, and leaderboard management, while its integration with PostgreSQL streamlined data operations.

Swagger UI: An open-source tool used to document and test the back-end APIs developed with Spring Boot. Swagger UI provided an interactive interface for exploring and validating API endpoints, streamlining development and ensuring robust API functionality.

Database

PostgreSQL: An open-source relational database management system used to store and manage game data, including user progress, scores, and maze configurations. PostgreSQL's scalability and ACID compliance ensured reliable data handling for leaderboard rankings and user accounts.

Development & Collaboration Tools

IntelliJ IDEA: A powerful integrated development environment (IDE) used primarily for Spring Boot and Java development. IntelliJ's robust debugging, code completion, and integration with Git/GitHub enhanced back-end development efficiency.

Git/GitHub: A version control system and hosting platform used for source code management and collaboration. Git facilitated iterative development through branching and merging, while GitHub served as the repository for code storage, version tracking, and team coordination.

Documentation Tools

Microsoft Word: A word processing tool used for documentation, including requirement specifications, design documents, and the project report. MS Word ensured professional formatting and organization of project deliverables.

Diagramming & Visualization Tools

Draw.io: Used to create system architecture diagrams, use case diagrams, and UI mockups. Its intuitive drag-and-drop interface made visual design easy and clear.

Mermaid.js: A JavaScript-based tool for generating diagrams and flowcharts using markdown-like syntax. It was used for rendering sequence diagrams and flow logic directly in documentation or markdown files, enabling developer-friendly visualization.

4.1.2 Implementation Details of Modules

1. **User Authentication Module**

   Responsibilities: Handles login, registration, JWT management, and session persistence.

   Key Components:

   typescript

   ```
   @Injectable()
   export class AuthService {
     // Observables for authentication state
     private authStatusSubject = new BehaviorSubject<boolean>(false);
     public authStatus$ = this.authStatusSubject.asObservable();
   ```

```
 // Core methods:
 // Checks localStorage for existing session
 checkAuth(): void { }

 // POST to /sign-in, handles JWT storage
 login(email: string, password: string): Observable<User> { }

 // POST to /register, navigates to OTP verification
 register(username: string, email: string, password: string): Observable<User> { }

 // Clears localStorage and resets observables
 logout(): void { }

 // Helper methods:
 // Stores token/user in localStorage
 private handleAuthentication(response: AuthResponse): void { }

 // POST to /otp/validate
 validateOtp(email: string, otp: string): Observable<any> { }
}
```

2. **Maze Generation Module (InsertMazeData Java)**

   Responsibilities: Pre-generates mazes at startup based on difficulty levels.

   Key Components:

```
typescript
@Configuration
public class InsertMazeData {
 // Core generation method:
 private List<Maze> generateMazesForCategory(LevelCategory category, int level) {
   // 1. Determines maze size and base score
   // 2. Generates maze structure using BFS path creation
   // 3. Adds random walls while ensuring solvability
```

```
  }

  // Algorithm helpers:
  // Uses BFS to create guaranteed start-end path
  private List<int[]> createPathWithBFS(...) { }

  // Places start (top-left) and end (bottom-right)
  private int[][] generateValidPositions(...) { }
}
```

3. **Game Logic Module (MazeService & ProgressService)**

   Responsibilities: Manages game state, player movements, and scoring.

   Key Components:

```
typescript
@Injectable()
export class MazeService {
  // Maze operations:
  // Converts backend data to playable grid
  convertMazeDataToCells(maze: Maze): MazeCell[][] {  }

  // Game mechanics:
  calculateScore(baseScore: number, timeElapsed: number, bfsUsed: number): number {
    // Score = baseScore - timePenalty - bfsPenalty
  }
}

@Injectable()
export class ProgressService {
  // Progress tracking:
  // POST to /user-progress/save
  saveProgress(mazeId: number, score: number, time: number, bfsUsed: number) {}
```

```
 // Completion checks:
 // Checks user's progress records
  hasCompletedLevel(mazeId: number): Observable<boolean> {   }
}
```

4. **BFS Module (MazeService)**

   Responsibilities: Pathfinding algorithm implementation.

   Key Components:

```
typescript
@Injectable()
export class MazeService {
  findPathBFS(maze: MazeCell[][], start: PlayerPosition, end: PlayerPosition):
PlayerPosition[] {
    // 1. Uses queue-based BFS traversal
    // 2. Tracks parent nodes for path reconstruction
    // 3. Returns shortest path or empty array
  }
}
```

5. **Leaderboard Module (LeaderboardService Angular + Java)**

   Responsibilities: Manages leaderboard data retrieval.

   Frontend:

```
typescript
@Injectable()
export class LeaderboardService {
 // GET /leaderboard/global with fallback
  getGlobalLeaderboard(): Observable<LeaderboardEntry[]> {  }

 // GET /leaderboard/rank/{userId}
  getUserRank(userId: number): Observable<number> {   }
}
```

Backend:

```
java
@Service
public class LeaderboardServiceImpl {
  public Page<LeaderboardResponse> getGlobalLeaderboard(Pageable pageable) {
    // Joins user_progress + maze + user tables
    // Orders by score DESC
  }

  // Calculates position from ordered scores
  public Integer getUserRank(Long userId) { }
}
```

4.2 Testing

4.2.1 Test Cases for Unit Testing

Table 4.1 Test Cases for User Login & Registration

SN	Test Case	Test Input	Expected Result	Outcome
1	Valid user registration	Email: ram@yopmail.com, Password: Ram@1234	Account created successfully, verification email sent	Pass
2	Registration with existing email	Email: ram@yopmail.com, Password: Ram@1234	Error message: "Email already exists"	Pass
3	Registration with invalid email format	Email: invalid-email, Password: Test123!	Error message: "Invalid email format"	Pass
4	Valid email verification	Click valid verification with OTP	Account status changed to verified	Pass
5	Invalid email verification	Expired/invalid OTP	Error message: "Invalid or expired OTP"	Pass

6	Valid user login	Email: ram@yopmail.com, Password: Ram@1234	User logged in, token created	Pass
7	Login with wrong password	Email: ram@example.com, Password: Ram@1	Error message: "Invalid credentials"	Pass
8	User logout	Click logout button	Token expired, clear local storage, redirected to landing page	Pass

Table 4.2 Test Cases for Game Functionality

SN	Test Case	Test Input	Expected Result	Outcome
1	Load easy level for public user	Click "Play Easy" button	Easy maze loads successfully	Pass
2	Load maze with server error	Simulate server failure	Error message: "Unable to load maze"	Pass
3	Use BFS hint on active maze	Click "Use BFS" button during gameplay	Optimal path highlighted on maze	Pass
4	Use BFS hint without active maze	Click "Use BFS" button on menu	Error message: "No maze active"	Pass
5	Save game progress (logged in user)	Click "Next Level" during gameplay	Progress saved to database	Pass
6	Save progress with server error	Click " Next Level" with DB connection failure	Error message: "Failed to save progress"	Pass
7	Access all levels (registered user)	Select easy/medium/hard difficulty	Respective difficulty maze loads	Pass

8	Access restricted levels (public user)	Attempt to access medium/hard levels	Redirect to login or access denied	Pass

Table 4.3 Test Cases for Profile Management

SN	Test Case	Test Input	Expected Result	Outcome
1	View user profile	Click "Profile" button	Profile data displayed correctly	Pass

Table 4.4 Test Cases for Leaderboard & Landing Page

SN	Test Case	Test Input	Expected Result	Outcome
1	View landing page	Access home URL	Landing page loads with game information	Pass
2	Landing page server error	Simulate server failure	Error message: "Unable to load page"	Pass
3	View leaderboard with data	Click "Leaderboard" button	Top players' scores and rankings displayed	Pass
4	View leaderboard without data	Click "Leaderboard" with empty database	Message: "No scores yet"	Pass

4.2.2 Test Cases for System Testing

Table 4.5 Test Cases for User Authentication Flow

SN	Test Case	Test Input	Expected Result	Outcome
1	Complete registration flow	Register → Verify email → Login	User successfully registered and logged in	Pass
2	Registration without email verification	Register → Attempt login without verification	Invalid Credentials Message shown	Pass

Table 4.6 Test Cases for Game Session Management

SN	Test Case	Test Input	Expected Result	Outcome
1	End-to-end gameplay (public user)	Play easy level from start to finish	Game completes successfully, score not recorded	Pass
2	End-to-end gameplay (registered user)	Play all difficulty levels	All levels accessible and playable	Pass
3	Save and resume game progress	Save progress → Logout → Login → Resume	Game continues from saved state	Pass
4	Cross-browser compatibility	Test on Chrome, Microsoft Edge	Consistent functionality across browsers	Pass

Table 4.7 Test Cases for Security & Access Control

SN	Test Case	Test Input	Expected Result	Outcome
1	Unauthorized access to protected routes	Direct URL access to /profile without login	Redirect to login page	Pass
2	Password security	Check password hashing	Passwords stored as secure hashes	Pass

Table 4.8 Test Cases for Integration & API Testing

SN	Test Case	Test Input	Expected Result	Outcome
1	Email service integration	User registration trigger	Verification email sent successfully	Pass
3	BFS algorithm accuracy	Complex maze with known solution	Hint shows mathematically optimal path	Pass
4	Leaderboard sorting accuracy	Multiple scores submission	Scores displayed in correct descending order	Pass

Chapter 5: Conclusion and Future Recommendations

5.1 Conclusion

In Conclusion, this project successfully delivers an engaging web-based platform that combines interactive maze-solving with educational value, effectively introducing players to the Breadth-First Search (BFS) algorithm. By offering 15 pre-generated mazes across Easy, Medium, and Hard difficulty levels, each with five sub-levels, the game provides diverse challenges for players of varying skill levels. The dual navigation approach: manual keyboard controls and limited BFS usage (up to three times per level), creates a dynamic experience that balances player with algorithmic learning. Registered users benefit from progress saving, score tracking, and a global leaderboard, fostering a competitive environment, while public users can access Easy levels, ensuring broad accessibility.

5.2 Lesson Learnt/Outcome

- Developing an educational game requires balancing fun gameplay with instructional content.
- Implementing pathfinding algorithms like BFS in a game context provides practical understanding of theoretical concepts.
- User account management and data persistence are crucial for retaining player engagement over time.
- Testing, especially for game logic and algorithm correctness, is vital to ensure a smooth user experience.

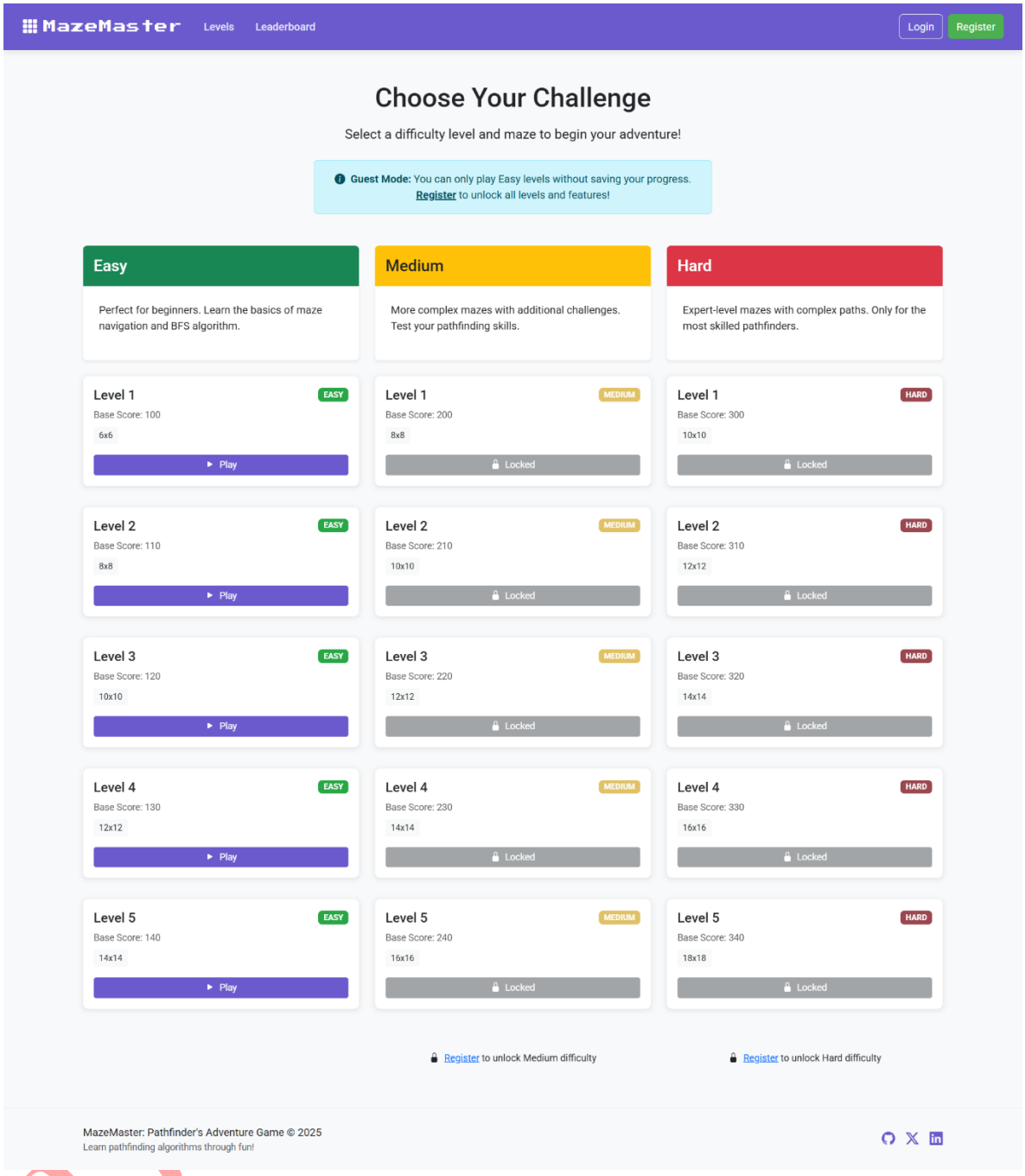
5.3 Future Recommendations

- Expand the game to include more pathfinding algorithms, such as DFS, A*, or Dijkstra's algorithm, to provide a broader educational scope.
- Introduce dynamic maze generation to increase replay ability and challenge.
- Add multiplayer features, such as competitive or cooperative maze solving.

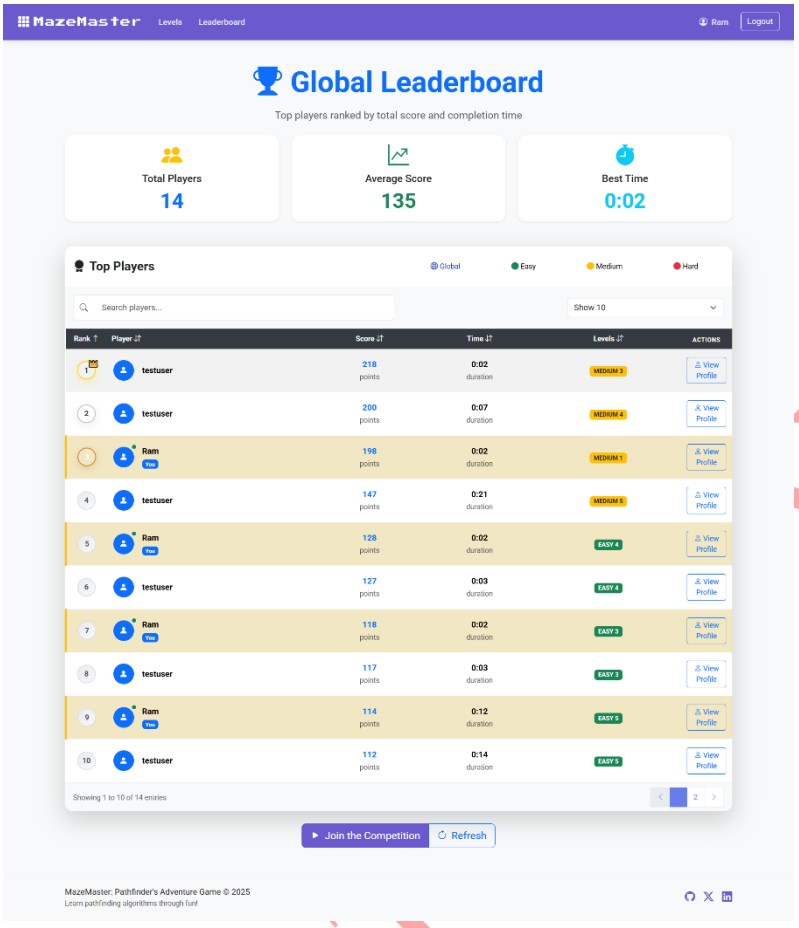
Appendices

Screen Shots

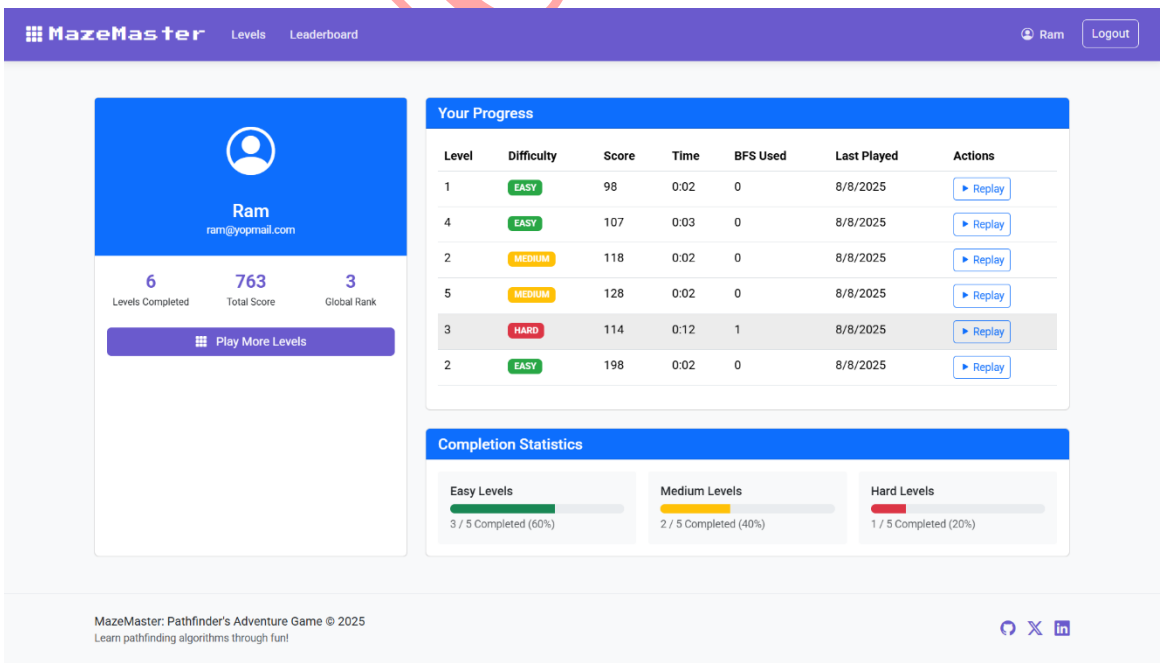
Landing Page


MazeMaster
Levels
Leaderboard
Login
Register
Choose Your Challenge
Select a difficulty level and maze to begin your adventure!
Guest Mode: You can only play Easy levels without saving your progress.
Register to unlock all levels and features!
Easy
Perfect for beginners. Learn the basics of maze navigation and BFS algorithm.
Medium
More complex mazes with additional challenges. Test your pathfinding skills.
Hard
Expert-level mazes with complex paths. Only for the most skilled pathfinders.
Level 1 EASY Base Score: 100 6x6 Play
Level 1 MEDIUM Base Score: 200 8x8 Locked
Level 1 HARD Base Score: 300 10x10 Locked
Level 2 EASY Base Score: 110 8x8 Play
Level 2 MEDIUM Base Score: 210 10x10 Locked
Level 2 HARD Base Score: 310 12x12 Locked
Level 3 EASY Base Score: 120 10x10 Play
Level 3 MEDIUM Base Score: 220 12x12 Locked
Level 3 HARD Base Score: 320 14x14 Locked
Level 4 EASY Base Score: 130 12x12 Play
Level 4 MEDIUM Base Score: 230 14x14 Locked
Level 4 HARD Base Score: 330 16x16 Locked
Level 5 EASY Base Score: 140 14x14 Play
Level 5 MEDIUM Base Score: 240 16x16 Locked
Level 5 HARD Base Score: 340 18x18 Locked
Register to unlock Medium difficulty
Register to unlock Hard difficulty
MazeMaster: Pathfinder's Adventure Game © 2025
Learn pathfinding algorithms through fun!

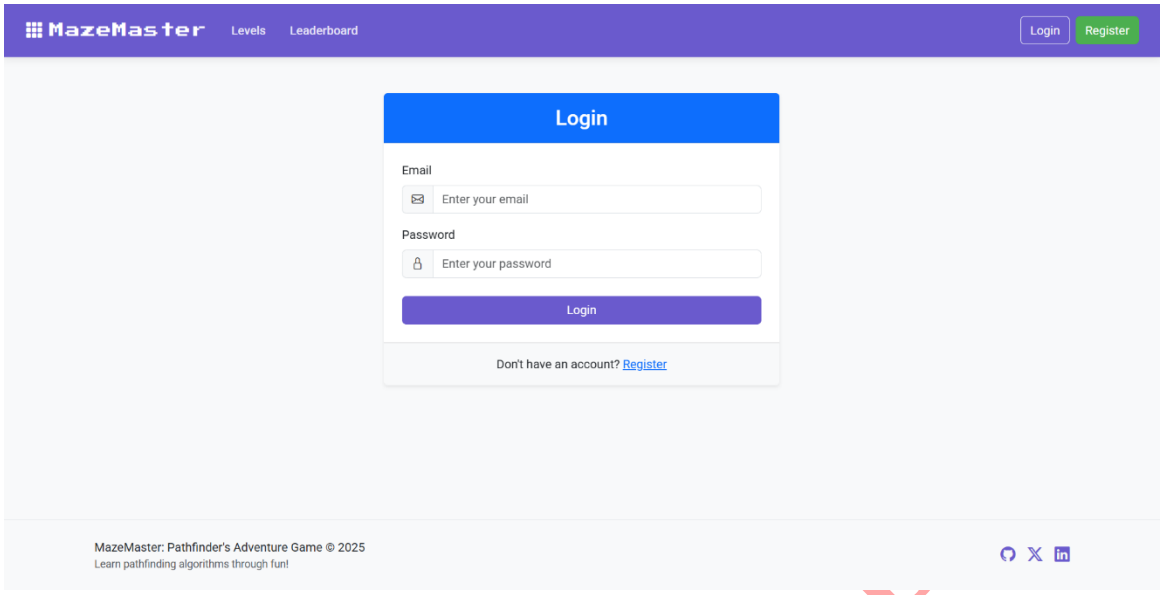
Level Page



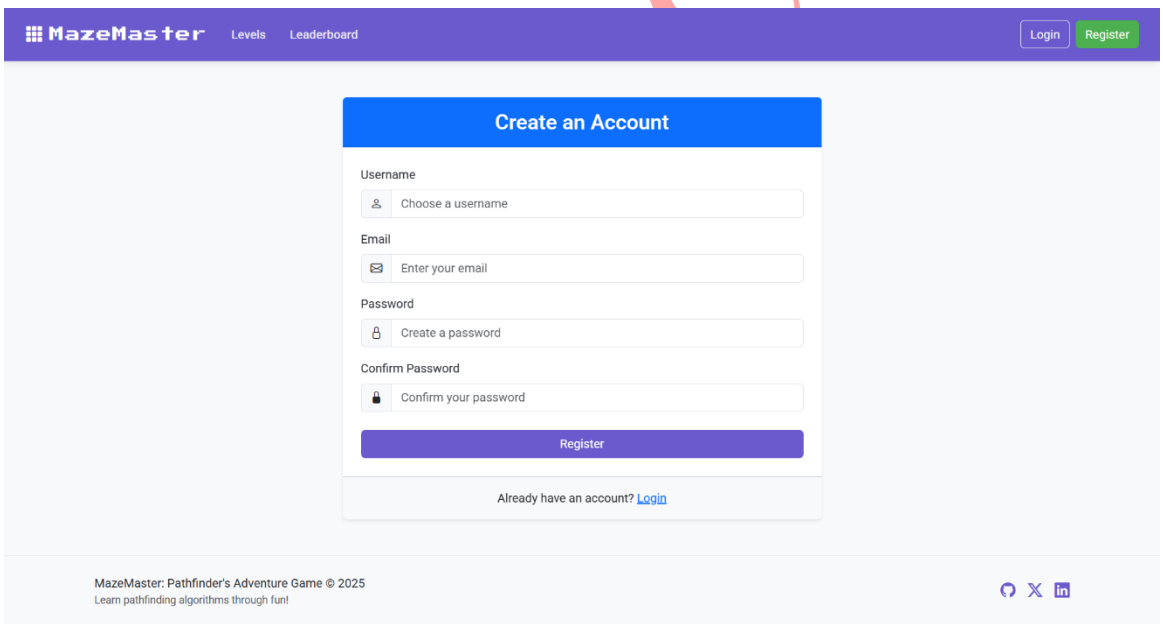
Leaderboard Page



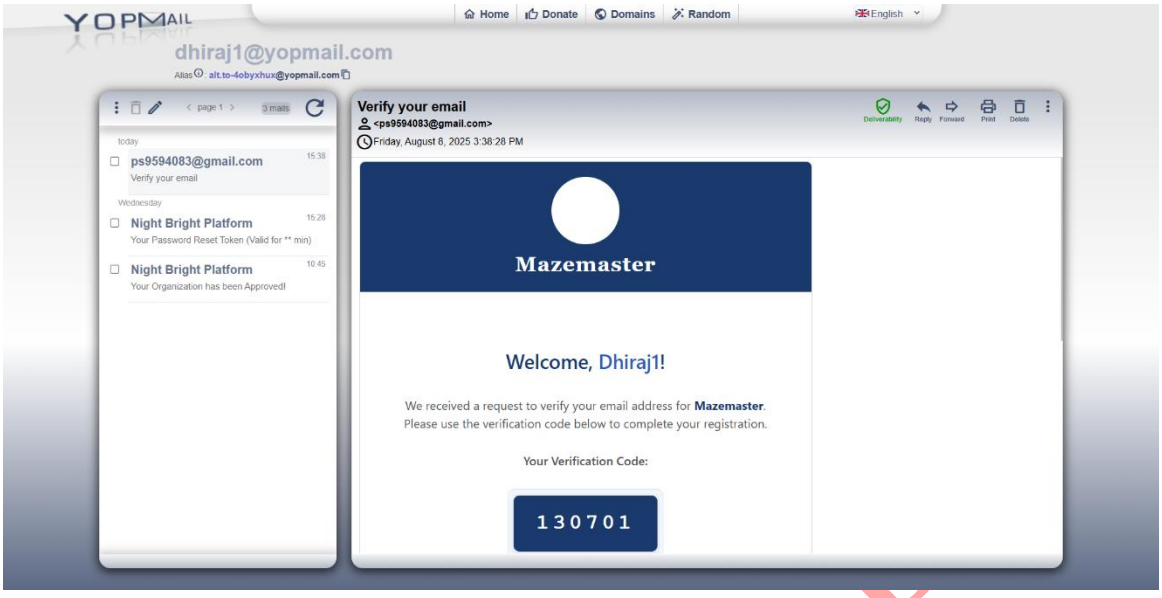
Profile Page



Login Page



Register Page



Email Verification Template

Source Code

```
package com.game.mazemaster_service;
import com.game.mazemaster_service.config.RSAKeyRecord;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.context.properties.EnableConfigurationProperties;

@EnableConfigurationProperties(RSAKeyRecord.class)
@SpringBootApplication
public class MazemasterServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(MazemasterServiceApplication.class, args);
    }
}
```

Others code are in the following github links:

Angular: https://github.com/Deejs2/mazemaster-ui

Spring Boot: https://github.com/Deejs2/mazemaster-service

References

[1] GeeksforGeeks, "Breadth First Search or BFS for a Graph," GeeksforGeeks, 23 July 2025. [Online]. Available: https://www.geeksforgeeks.org/breadth-first-search-or-bfs-for-a-graph/. [Accessed 21 April 2025].

[2] Pardede, S. a. Athallah, F. a. Huda, Y. a. Zain and Fikri, "A Review of Pathfinding in Game Development," *[CEPAT] Journal of Computer Engineering: Progress, Application and Technology,* vol. 1, no. CC BY-NC 4.0, p. 47, May 2022.

[3] A. R. R. Putra, I. R. I. M. Rasyid and D. Ramadana, "Puzzle game solving with breadth first search algorithm," *IOP Conf. Series: Journal of Physics: Conf. Series,* vol. 1402, no. 6, p. 066040, December 2019.

[4] I. E. Team, "What is Iterative Methodology," Indeed, 02 July 2024. [Online]. Available: https://uk.indeed.com/career-advice/career-development/what-is-iterative-methodology. [Accessed 14 June 2025].

[5] T. Cant, "Introduction to Pathfinding with Mazes and Breadth-first Search," Tom Cant, 13 September 2021. [Online]. Available:https://tomcant.dev/posts/2021/09/introduction-to-pathfinding-with-mazes-and-breadth-first-search/. [Accessed 23 July 2025].

[6] Luthfisauqi, "Artificial Intelligence Search Problem: Solve Maze using Breadth First Search (BFS) Algorithm," Medium, 01 March 2024. [Online]. Available: https://medium.com/@luthfisauqi17_68455/artificial-intelligence-search-problem-solve-maze-using-breadth-first-search-bfs-algorithm-255139c6e1a3. [Accessed 18 July 2025].

[7] GeeksforGeeks, "Shortest path in a Binary Maze," GeeksforGeeks, 23 July 2025. [Online]. Available: https://www.geeksforgeeks.org/dsa/shortest-path-in-a-binary-maze/. [Accessed 12 July 2025].